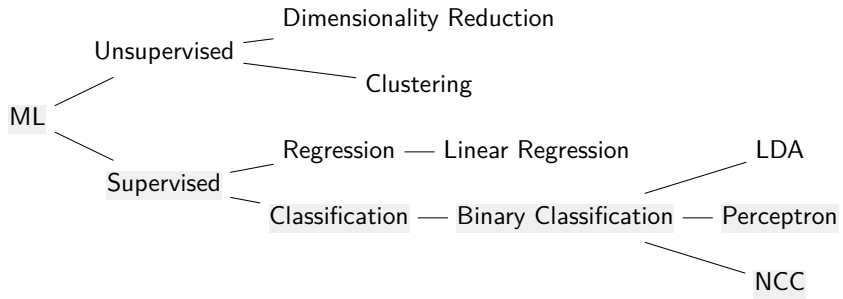
Classification



Binary Classification

Machine Learning Basics

Based on knowledge (dataset) we want to answer or solve a problem.

Notation hint

Here $\mathbf{w}$ and $\mathbf{x}$ (bold letters) denote vectors in $\mathbb{R}^d$. We usually use d for the dimension of a datapoint and n for the number of datapoints.* Consequently, we denote $X \in \mathbb{R}^{n \times d}$ as the matrix containing the datapoints $\mathbf{x}_i$ (i.e a list of datapoints).

$$X = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \dots & \mathbf{x}_n \end{bmatrix}$$

y_i refers to the label corresponding to $\mathbf{x}_i$.

*not always consistent, but in these notes.

Example

We have a dataset comprising of 100 PNG images with a resolution of 8×8 and corresponding labels `dog` and `cat`. Each image could be represented as a 64-dimensional vector $\mathbf{x} \in \mathbb{R}^{64}$, and each label y could be represented by 0 or 1.

Binary Classification

Based on a dataset containing data points with class labels belonging to two classes, we want to predict the class labels of future unlabeled data points. Consider this example of a two-dimensional dataset.

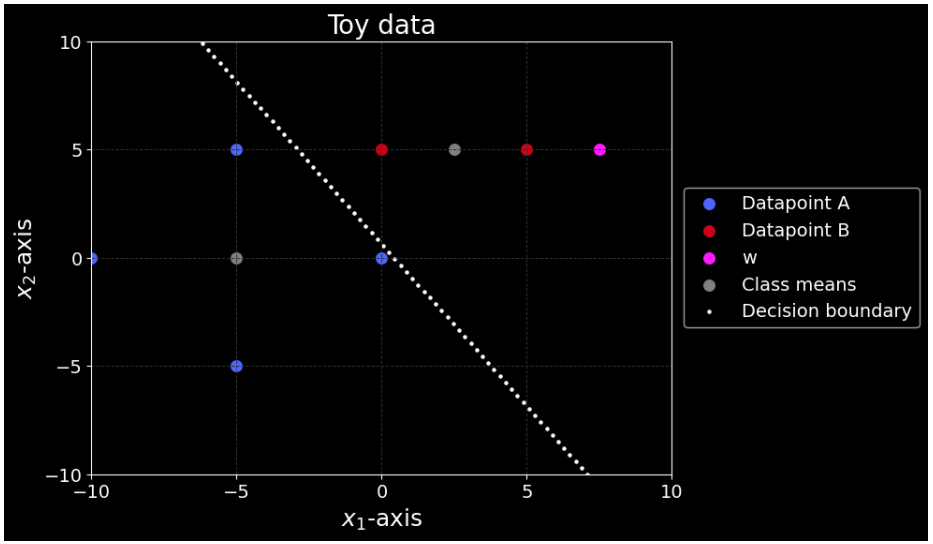


Figure 1: Plot of a two-dimensional dataset containing two classes (Blue A and Red B). The axes represent the first and second entries of each data point. The weight vector $\mathbf{w}$ is shown as a magenta point. The decision boundary is the set of points satisfying $\mathbf{w}^\top \mathbf{x} - \beta = 0$. Below is how we can represent both datasets in Python.

```
A = np.array([[0,0],[-5,5],[-10,0],[-5,-5]])
B = np.array([[0,5],[5,5]])
```

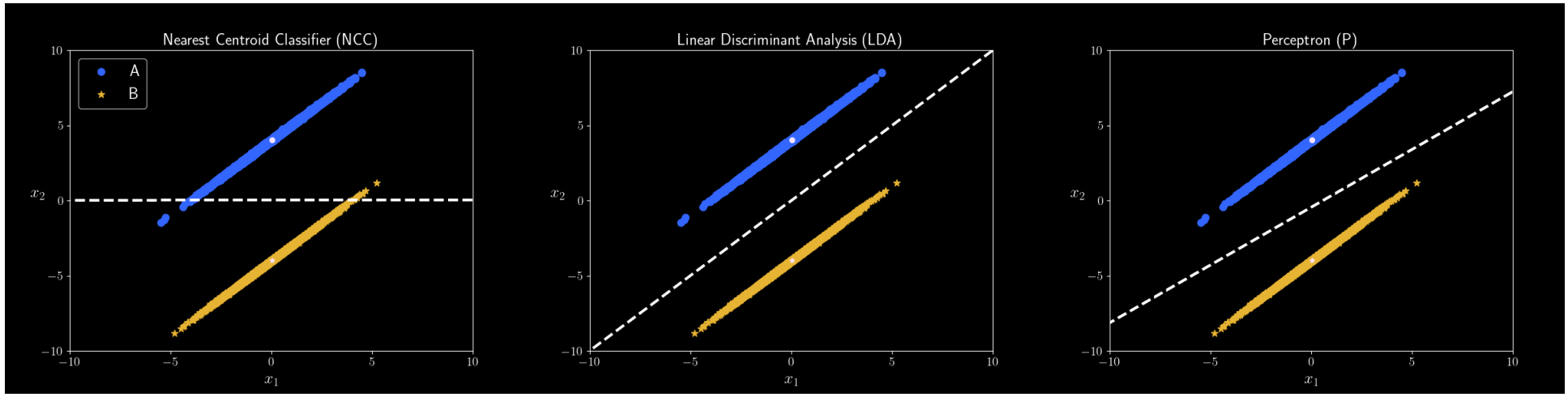


Figure 1: Each plot shows the same dataset containing two-dimensional datapoints $x \in \mathbb{R}^2$ belonging to two classes (blue and orange). The white dashed line in each plot shows the decision boundary which resulted from NCC, LDA (later) or the Perceptron. Note that the perceptron can have multiple outcomes for the same dataset.

In general we can write a binary classification model as

Binary classification model

Let $\mathbf{w}, \mathbf{x} \in \mathbb{R}^n$ and $\beta \in \mathbb{R}$ then

$$sign(\mathbf{w}^\top \mathbf{x} - \beta)$$

returns either -1 or 1.

We call the set of points $\mathbf{x} \in \mathbb{R}^n$ that fullfill the equation $\mathbf{w}^\top \mathbf{x} = 0$ decision boundary.We will learn 3 methods to derive $\mathbf{w}, \mathbf{x}$ and β for binary classification. In this chapter Nearest Centroid Classifier (NCC) and Perceptron. Later, Linear discriminant analyis (LDA). The result of NCC and LDA will always be the same on the same dataset because the result is given by a formular.

Nearest Centroid Classifier

NCC formular for $\mathbf{w}$ and β

Given Dataset with datapoints $\mathbf{x} \in \mathbb{R}^n$, denote x_+, x_- datapoints belonging to 0,1

$$\mathbf{w} = \bar{\mathbf{x}}_- - \bar{\mathbf{x}}_+ \quad \text{and then} \quad \beta = \frac{1}{2}(\mathbf{x}_-^\top \mathbf{x}_- - \mathbf{x}_+^\top \mathbf{x}_+)$$

Example

Let $\begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}0\\2\end{bmatrix} \in \mathbb{R}^2$ be in class - and $\begin{bmatrix}1\\1\end{bmatrix} \in \mathbb{R}^2$ be in class $+$. Then we can calculate $\mathbf{w}$ by

$$\frac{1}{2}\left(\begin{bmatrix}0\\0\end{bmatrix} + \begin{bmatrix}0\\2\end{bmatrix}\right) = \begin{bmatrix}0\\1\end{bmatrix} = \bar{\mathbf{x}}_- \text{ and } \begin{bmatrix}1\\1\end{bmatrix} = \mathbf{x}_+ \text{ then } \mathbf{w} = \begin{bmatrix}-1\\0\end{bmatrix} = \begin{bmatrix}0\\1\end{bmatrix} - \begin{bmatrix}1\\1\end{bmatrix}$$

and β can be calculated by

$$\beta = \frac{1}{2}(1-2) = -\frac{1}{2}$$

Perceptron

Unlike NCC, Perceptron uses an iterative algorithm to update $\mathbf{w}, \beta$ until there are no missclassified points left. For example look at the right side of this page.

Algorithm 1 Perceptron Learning Algorithm

Input: Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$, $y_i \in \{-1, 1\}$ and step size $\eta > 0$.
Output: Weight vector $\mathbf{w}$, bias β

```
Initialize w ← random ∈ ℝ^d, b ← random ∈ ℝ
repeat
    # Loop through each training example
    for each (x_i, y_i) do
        # Check if x_i is missclassified
        if y_i(w^⊤ x_i + β) ≤ 0 then
            w ← w + η y_i x_i
            β ← β + η y_i
until no misclassifications
```

Because of random initialization of $\mathbf{w}$ and β the perceptron algorithm can result in different decision boundaries.

Task

How can we modify the algorithm to only use modified $\mathbf{x}$ and $\mathbf{w}$?

Solution Augmented Notation

Given $\mathbf{x}, \mathbf{w} \in \mathbb{R}^d$ then $\tilde{\mathbf{x}}, \tilde{\mathbf{w}} \in \mathbb{R}^{d+1}$. We modify the last entry of $\mathbf{x}$ to be 1 and the last entry of $\mathbf{w}$ as β.

$$\tilde{\mathbf{x}}_i^\top \tilde{\mathbf{w}}_i = \tilde{\mathbf{x}}_{i1} \cdot \tilde{\mathbf{w}}_{i1} + \tilde{\mathbf{x}}_{i1} \cdot \tilde{\mathbf{w}}_{i1} + \ldots \tilde{\mathbf{x}}_{id} \cdot \tilde{\mathbf{w}}_{id} + 1 \cdot \beta = \mathbf{x}^\top \mathbf{w} + \beta$$

Perceptron loss function; Perceptron Error

The function

$$\mathcal{L}_{\mathcal{P}} : \mathbb{R}^d \to \mathbb{R}, \quad w \mapsto -\sum_{m \in \mathcal{M}} w^\top x_m y_m$$

is called the **Perceptron loss functions**

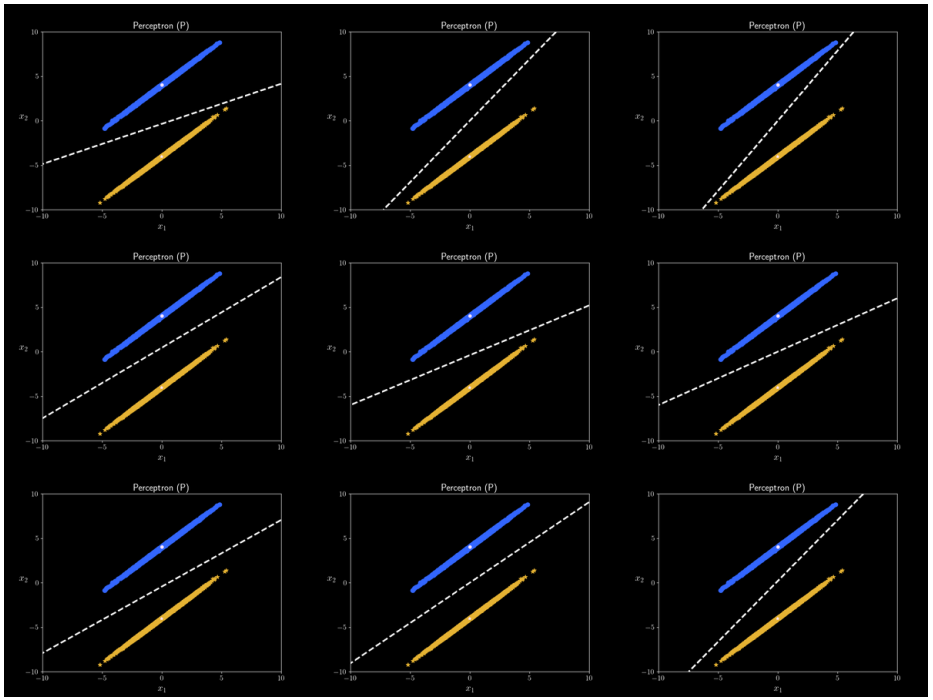


Figure 2: This figure shows the same two-dimensional dataset consisting of two classes (blue and orange) and the result of the perceptron learning algorithm nine times. In each plot a different decision boundary (White dashed line) is depicted. This is a result of a random starting point. Different random initialized $\mathbf{w}$ and β result in a different set of $\mathcal{M}$ (Set of indices of missclassified points).

In lecture 6 we will learn that the idea of the perceptron can be extended by creating a whole network of perceptrons. The perceptron is a building block for the multilayer perceptron (Neural network).

Linear Discriminant Analysis

Data often follows an underlying distribution, and most commonly a Gaussian distribution. Linear Discriminant Analysis tries to exploit this. For more stochastic math background, go to the preliminary math section at the beginning of these notes (Under Construction).

Covariance Matrix

The covariance matrix contains information about the covariance of each possible combination of random variables for a dataset in $\mathbb{R}^{d\times d}$. It is definied as

Covariance Matrix Σ_X

$$\Sigma_X = \begin{bmatrix} Var(X_1) & \dots & Cov(X_1, X_d) \\ \vdots & \ddots & \vdots \\ Cov(X_d, X_1) & \dots & Var(X_d) \end{bmatrix} \in \mathbb{R}^{d\times d}$$

We can we can estimate it numerically by

Empirical Covariance Matrix $\hat{\Sigma}_X$ Formular

$$\hat{\Sigma}_X = \frac{1}{n}(X - \bar{X})(X - \bar{X})^\top$$

where $X \in \mathbb{R}^{d\times n}$ datamatrix containing datapoints in each column, $\bar{X} \in \mathbb{R}^{d\times n}$ containing the mean $\mu \in \mathbb{R}^d$ of all datapoints in each column

$$\bar{X} = \begin{bmatrix} | & | & & | \\ \mu & \mu & \dots & \mu \\ | & | & & | \end{bmatrix}.$$

Note that the formular above can be used for non centered data, since we substract the mean of each datapoint first.

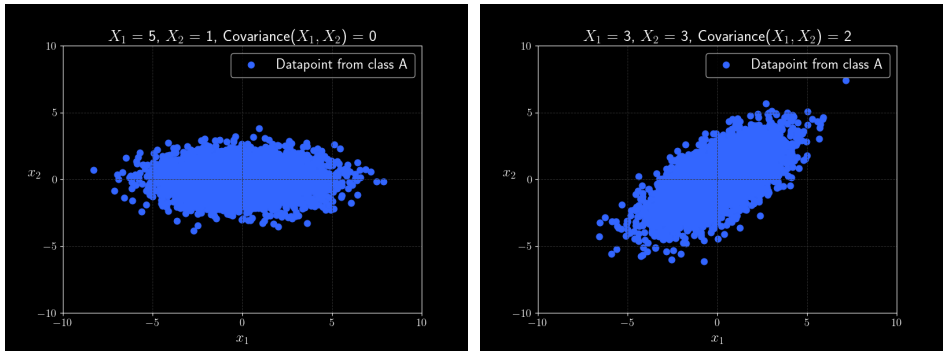


Figure 3: This figure shows the two datasets with datapoints in $\mathbb{R}^2$. On the left side the emperical covariance matrix is $\begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$ and on the right side $\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$

LDA formular for $\mathbf{w}$ and β

given a dataset $X \in \mathbb{R}^{d\times n}$ and class means $\mathbf{x}_+, \mathbf{x}_- \in \mathbb{R}^d$

$$\mathbf{w} = {S_w}^{-1}(\mathbf{x}_+ - \mathbf{x}_-)$$
$$\beta = \frac{1}{2}\mathbf{w}^T(\mathbf{x}_+ + \mathbf{x}_-) + \log\left(\frac{n_-}{n_+}\right).$$

where S_w is

$$S_w = S_+ + S_- = \frac{1}{n_+}(X_+ - \bar{X}_+)(X_+ - \bar{X}_+)^\top + \frac{1}{n_-}(X_- - \bar{X}_-)(X_- - \bar{X}_-)^\top$$

and X_+ contains datapoints labeled $+$.

Cognitive Algorithms Tutorial Notes Hannah Louisa Boldt
Notes are not complete and can contain typos. If you spot any mistakes, typos or have any questions please email me at `h.boldt@campus.tu-berlin.de`*!*